

DETAILED DESIGN

Elastic Enclave HSM — EJBCA PKI + SPIRE

Zero-Trust

Attested, autoscaling, FIPS 140-3 Level 1 certificate & workload-identity signing on commodity EC2

NitroEnclave · signer gateway

Validated on AWS · 2026

Status: validated single-box; NLB scaling planned

Executive overview

The Elastic Enclave HSM turns one in-house signer gateway into an elastic, FIPS 140-3 Level 1 hardware security module that serves two independent product paths from the same fleet: EJBCA certificate issuance and SPIRE workload-identity (SVID) signing. The CA/ICA signing key is released only into an attested AWS Nitro Enclave via KMS and never leaves that boundary — the parent EC2 host, the gateway, and every caller ever see only digests going in and signatures coming out. This delivers HSM-grade, post-quantum-ready key custody at spot-EC2 economics, scaling to load instead of to peak, and has been validated end-to-end on real AWS.

Throughput is framed honestly at two levels. The raw signing ceiling — the HSM primitive that turns a random digest into a signature — peaks at ~8,600 ecdsa-p384 signs/sec on half a c9g.2xlarge and is never the bottleneck. Real product issuance is service-bound: ~895 certs/sec through EJBCA and ~236 SVIDs/sec through the SPIRE server. The signer therefore carries roughly 10–36× headroom over both issuance tiers; you scale EJBCA nodes and SPIRE servers, not the signer.

1 FIPS 140-3 — Level 1 and Level 3

The design is deliberate about which assurance level does which job. Two levels of the FIPS 140-3 standard are relevant.

FIPS 140-3 Level 1

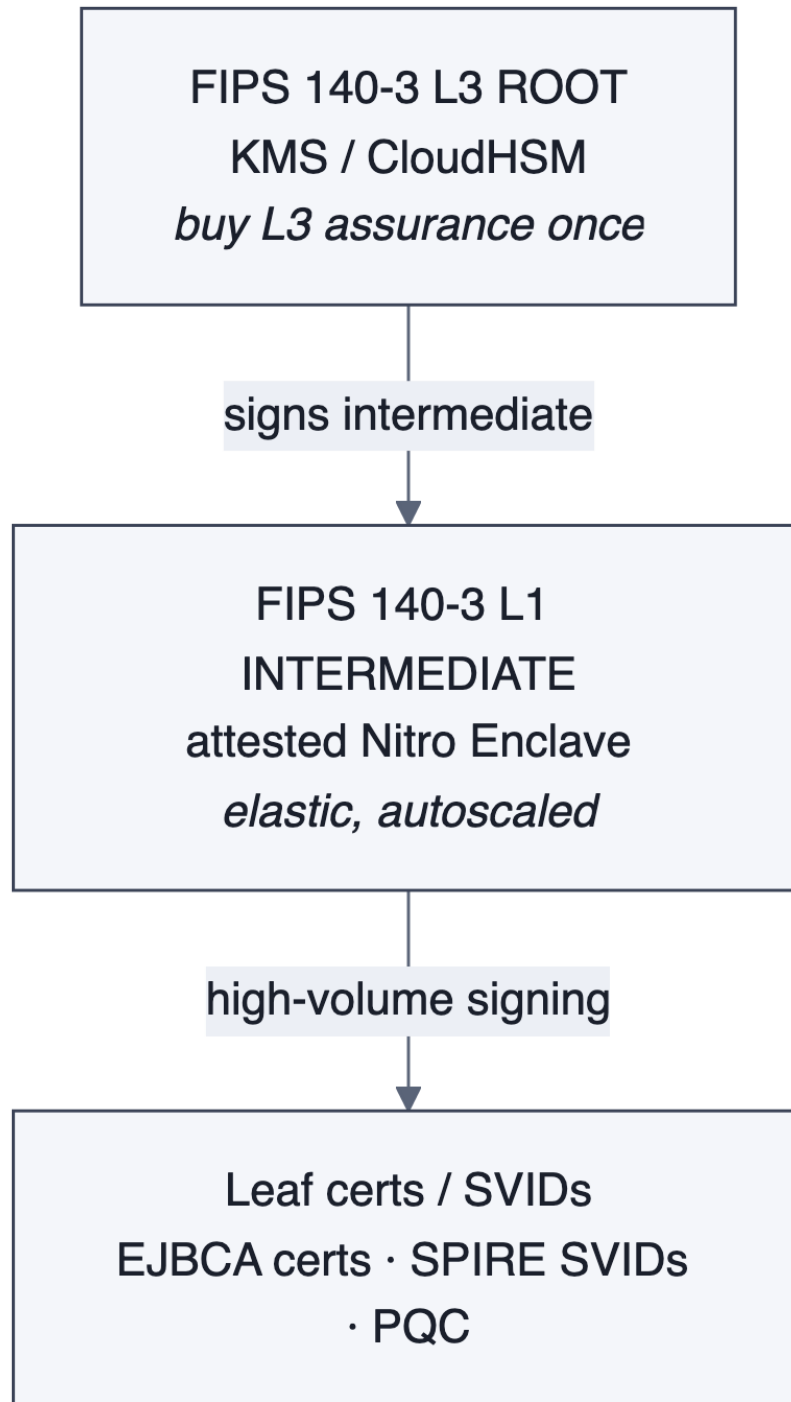
Production-grade validated crypto (FIPS-approved algorithms). No physical tamper hardware required. The enclave adds hardware-isolated, operator-inaccessible memory + cryptographic attestation of the exact image that may hold the key — a strong isolation boundary at L1 economics.

FIPS 140-3 Level 3

Adds tamper-evident/responsive physical security, identity-based operator authentication, and physical separation of plaintext critical security parameters. This is the CloudHSM / dedicated-HSM tier.

Tiered assurance model

An L3 root (KMS/CloudHSM) can anchor the trust chain and sign the intermediate; the high-volume leaf/SVID signing runs at L1 inside the attested enclave. You buy L3 assurance once for the root, and scale L1 signing elastically.

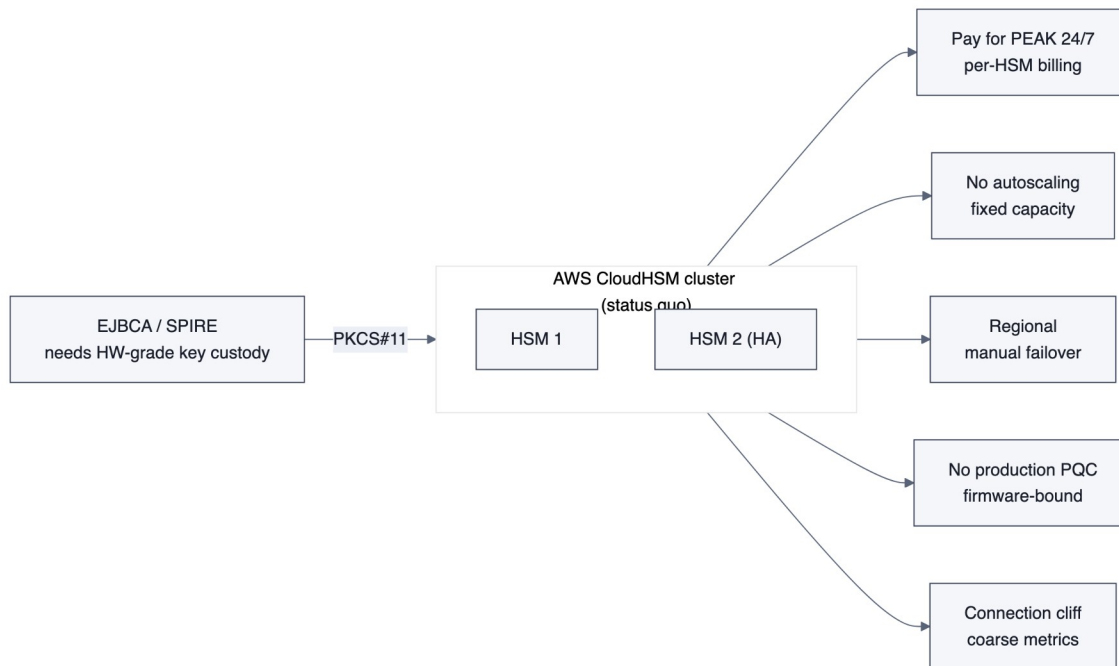


Tiered assurance: an L3 root anchors the chain; elastic L1 enclaves do the high-volume signing.

2 Problem statement — why CloudHSM does not fit high-scale identity

A dedicated HA CloudHSM cluster is the incumbent answer for HSM-grade custody, but it is a poor fit for high-volume, elastic workload identity for six concrete reasons.

- **Cost of HA at scale** — An HA CloudHSM cluster bills 24/7 per HSM for peak capacity — expensive whether you sign 10 or 10,000 TPS.
- **No autoscaling** — Fixed capacity. You provision for peak and pay for it continuously; there is no scale-to-load.
- **Regional, manual failover** — Cross-region resilience is a manual, operational burden.
- **No production post-quantum** — No ML-DSA / ML-KEM / hybrid signing in the firmware today.
- **Firmware-bound mechanisms** — Signing is limited to built-in mechanisms — no hybrid/composite construction.
- **Infra-level metrics + connection cliff** — Observability is coarse; a hard connection limit creates a throughput cliff under burst.



Status quo: a fixed HA CloudHSM cluster billed for peak.

3 Proposed solution — an elastic enclave HSM

Replace the fixed HSM cluster with an attested Nitro Enclave signer behind an autoscaling fleet. One signer serves EJBCA and SPIRE, holds the key only inside the enclave boundary, and scales elastically. Four pillars define it.

Elastic — pay for load

No dedicated HSM. Spot EC2 + KMS, autoscale to min when idle. Linear scale-out: add boxes -> +throughput, behind a multi-AZ NLB.

HSM-grade custody (FIPS 140-3 L1)

The CA/ICA key is released only into an attested enclave via KMS and never leaves that boundary. Parent host and operators never see plaintext key.

PQC-native

ML-DSA-65, ML-KEM-768/1024 and hybrid (P-384+ML-DSA) signing — in software, today, not waiting on HSM firmware.

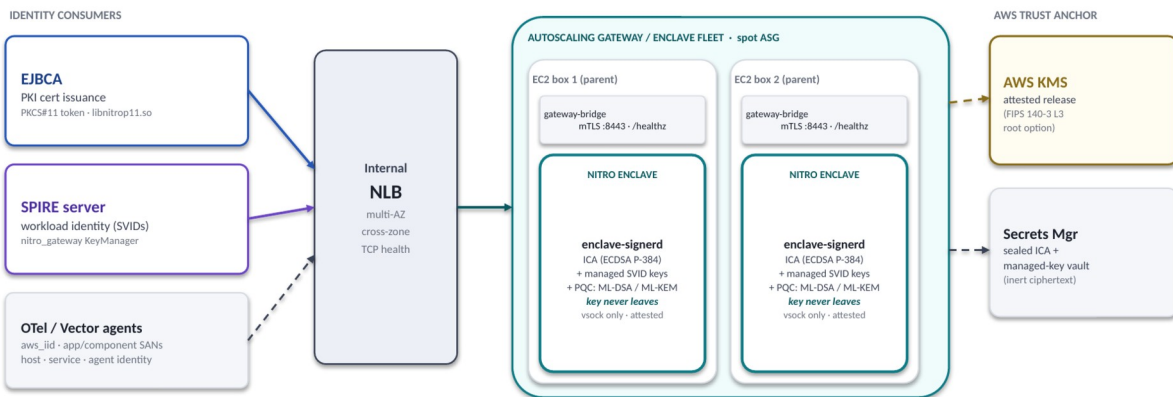
Enterprise resilience

Enclave-aware health reporting; a wedged enclave -> 503 -> NLB -> ASG auto-replaces. Self-heals under chaos, validated on real spot infra.

ELASTIC ENCLAVE HSM

Solution architecture — attested signer behind an autoscaling fleet

One elastic enclave HSM serves EJBCA PKI and SPIRE zero-trust — key never leaves the enclave



Unified architecture — one attested signer behind an autoscaling fleet serving EJBCA and SPIRE.

4 Architectures validated

Three usage patterns were built and load-tested on real AWS against the same enclave signer. Each is a distinct consumer of the sign-only API; all three coexist on one running gateway with zero interference.

A • EJBCA PKI issuance

EJBCA uses a stock PKCS#11 token (libnitrop11.so). Every C_Sign forwards over mTLS to the in-enclave signer holding the KMS-released ICA. No EJBCA rebuild.

Measured result

Measured ~895 certs/sec/node (ecdsa-p256, 0 errors); EJBCA-bound, scales with EJBCA nodes.

B • SPIRE zero-trust identity

SPIRE server uses a nitro_gateway KeyManager plugin -> the enclave signs every X509/JWT SVID. SPIRE server can also be an EJBCA child ICA (UpstreamAuthority) for a central trust root.

Measured result

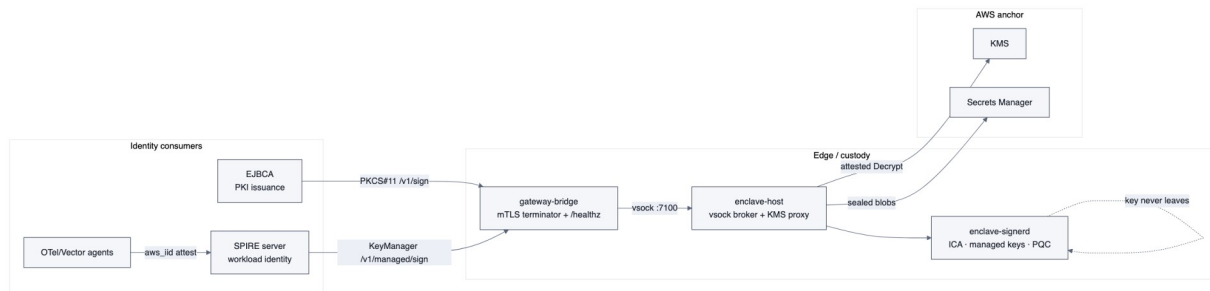
Measured ~236 SVIDs/sec (agent->server->gateway); SPIRE-server-bound, scales with servers.

C • Raw signing / lightweight HSM

Direct /v1/sign — a generic elastic HSM. Also the way to benchmark EJBCA itself without CloudHSM cost/complexity: remove the HSM as a variable and measure EJBCA's own ceiling.

Measured result

Raw ceiling ~8.6k signs/sec/box (ecdsa-p384) -> ~48k/sec across a small fleet.



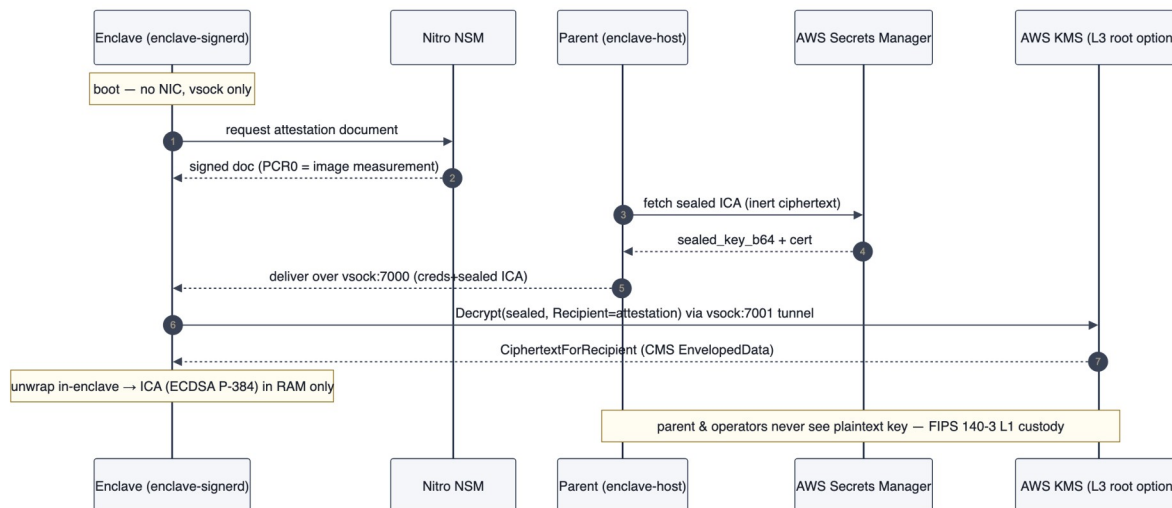
Component view — consumers, custody edge, AWS anchor.

5 Attested key custody

The enclave has no NIC and no IMDS. At boot it pulls instance credentials and the sealed ICA from the parent enclave-host over vsock, then asks the Nitro Security Module (NSM) for a signed attestation document carrying its measured PCRO and public key. It stands up a loopback→vsock forwarder so its own TLS session to KMS stays end-to-end through a transparent tunnel — the parent never terminates it.

The enclave then calls `kms:Decrypt` on the sealed ICA with Recipient set to the attestation document. The KMS key policy binds release to the exact measured PCRO, so only this enclave image can obtain the key; KMS returns CMS EnvelopedData. The enclave unwraps the CMS in-enclave, and the ECDSA P-384 intermediate CA key materializes only in enclave RAM — it never touches disk and never crosses the vsock boundary as plaintext.

Managed keys (the SPIRE CA keys) are generated inside the enclave and sealed with an attested `GenerateDataKey`: the private key is AES-256-GCM-wrapped in-enclave and only inert ciphertext is shipped to the parent, which persists it in a Secrets Manager vault (an `id→blob` map). Any attested enclave with the same PCRO can re-release and restore that key, making the enclave tier stateless without ever exposing plaintext key material.



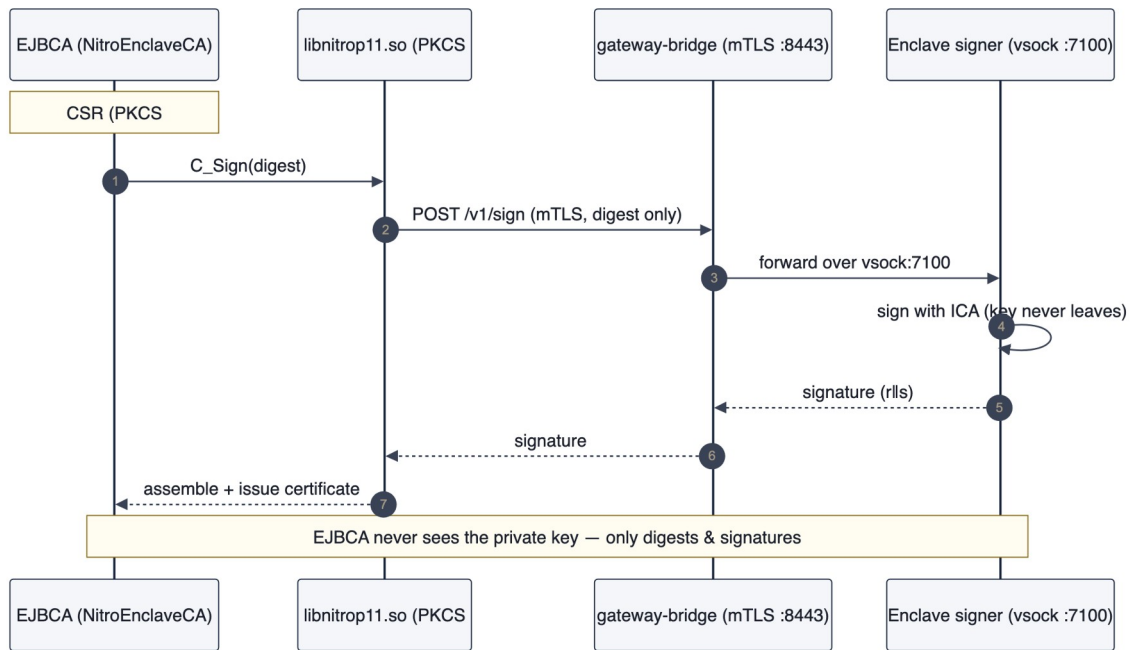
Attested release — the ICA is decrypted only inside the measured enclave image.

FIPS 140-3 L1 custody

The parent host and operators never see plaintext key material. mTLS terminates on the parent, but only digests go in and signatures come out — every signature happens inside the enclave.

6 EJBCA certificate issuance flow

EJBCA consumes the enclave through a stock PKCS#11 provider — libnitrop11.so — loaded as a NitroEnclaveCA CryptoToken, so no EJBCA rebuild is required. When EJBCA builds a TBSCertificate it hashes it to a digest and calls C_FindObjects by ICA label, then C_Sign. The .so POSTs /v1/sign {key_label, alg=ecdsa-p384, digest_b64} to gateway-bridge on :8443 over mTLS; the bridge forwards it over vsock :7100 into the enclave, which signs with the attested-released ICA and returns the raw r||s PKCS#11 form (96 bytes for P-384). The signature returns up the same path and EJBCA assembles the certificate. EJBCA never sees the private key — only digests out and signatures in.

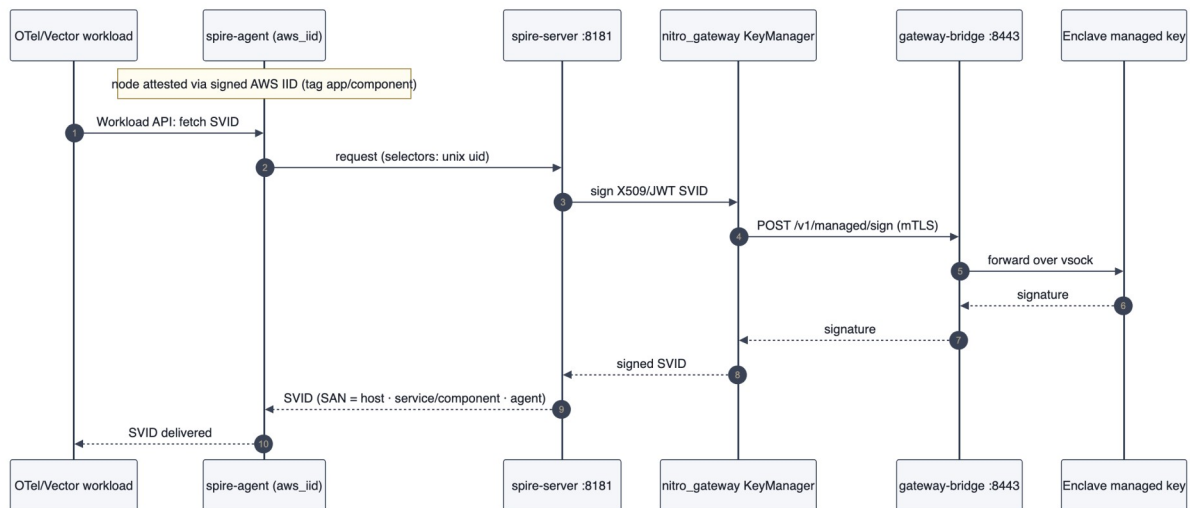


EJBCA issuance — every C_Sign forwarded to the in-enclave ICA.

7 SPIRE workload-identity flow

SPIRE consumes the same enclave through a nitro_gateway KeyManager plugin. When the SPIRE server mints an X509- or JWT-SVID it validates the request, builds the SVID TBS, and calls the KeyManager's SignData; the plugin POSTs /v1/managed/sign to the gateway over mTLS through a pooled client with a hard timeout. The bridge forwards over vsock :7100 and the enclave signs with the enclave-held managed CA key, returning ASN.1 DER (what SPIRE's crypto.Signer expects — distinct from EJBCA's raw r||s form). Agents hold no CA key; every SVID is one enclave round-trip on the server.

The SPIRE server can also be placed under a central trust root: with an EJBCA UpstreamAuthority the server still generates its intermediate CA key inside the enclave but submits its CSR to EJBCA, which signs the cert — chaining all SVIDs to the pinnable EJBCA root while custody of the signing key stays in the enclave. The self-rooted variant is today's validated default.



SPIRE SVID issuance — signed in the enclave, SAN scoped host/service/agent.

8 Use case — OpenTelemetry agent auth with component-level zero trust

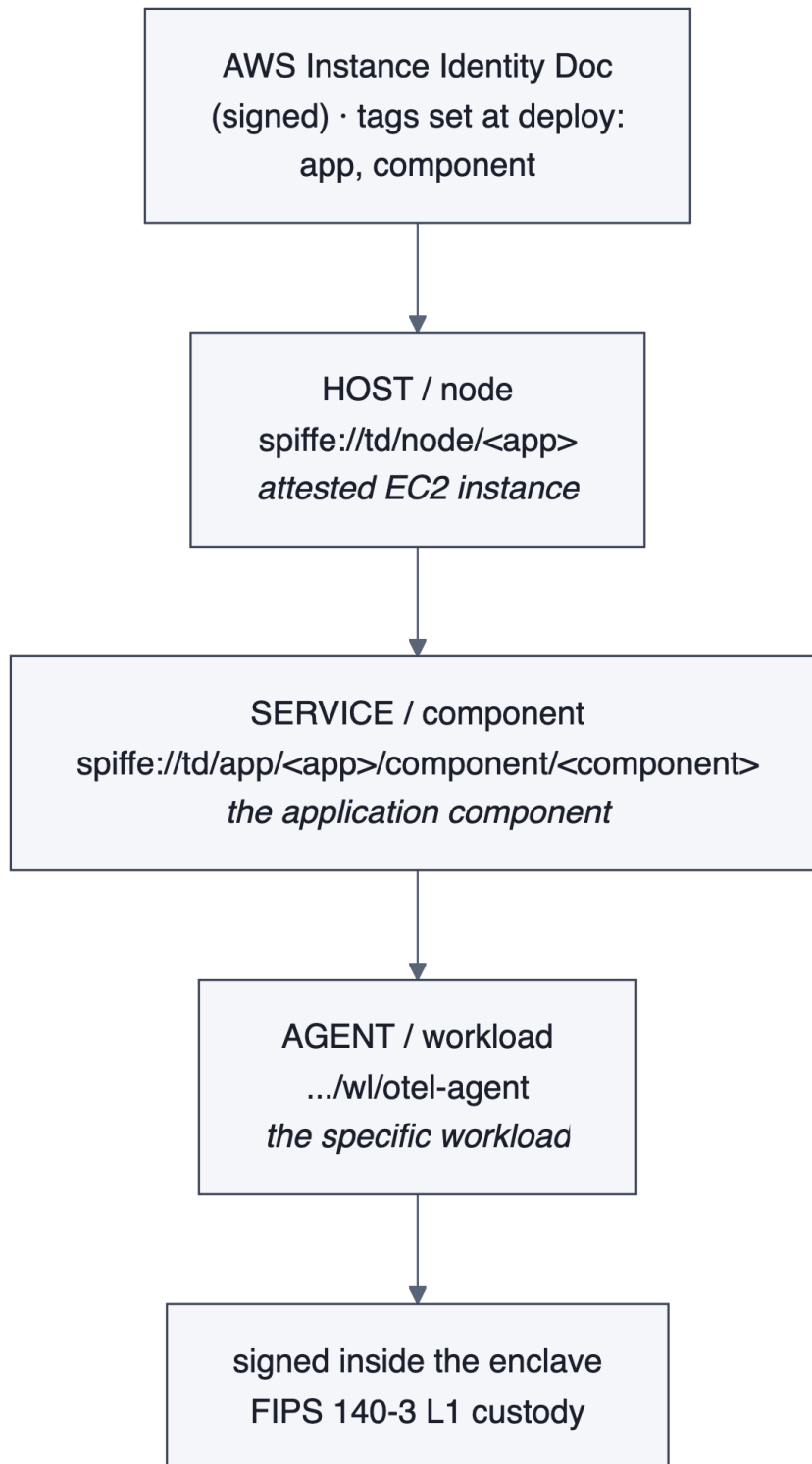
Use case — OpenTelemetry agent auth with component-level zero trust

The headline use case: cryptographic, component-scoped identity for a fleet of telemetry agents, minted from deploy-time tags and signed in the enclave.

Fleet OpenTelemetry (and Vector) agents must authenticate to the pipeline. Host-only identity is not enough for zero trust — two services on the same box must be distinguishable. At deploy time each instance is tagged `app=<app>` and `component=<component>`. SPIRE's `aws_iid` node attester reads those tags from the signed AWS Instance Identity Document and mints a SPIFFE ID whose SAN encodes identity at three levels:

Level	SPIFFE SAN	Meaning
Host / node	<code>spiffe://td/node/<app></code>	the attested EC2 instance (<code>aws_iid</code> , <code>tag:app</code>)
Service / component	<code>spiffe://td/app/<app>/component/<component>/...</code>	the application component — set from deploy-time tags, not inferred
Agent / workload	<code>.../wl/otel-agent</code>	the specific workload on the node (unix selector)

Every SVID is signed inside the enclave. Result: OTEL agents authenticate with a cryptographic identity scoped to host + service-component + workload — true zero trust, and the signing key never leaves FIPS 140-3 L1 custody. `app/component` populated during deployment, enforced at the identity layer, and centrally rooted through EJBCA.



Identity at three levels, minted from deploy-time app/component tags via aws_iid.

Why it is zero trust

app and component are populated during deployment, read from the signed AWS Instance Identity Document (aws_iid), enforced at the identity layer, and centrally rooted through EJBCA — two services on the same host are cryptographically

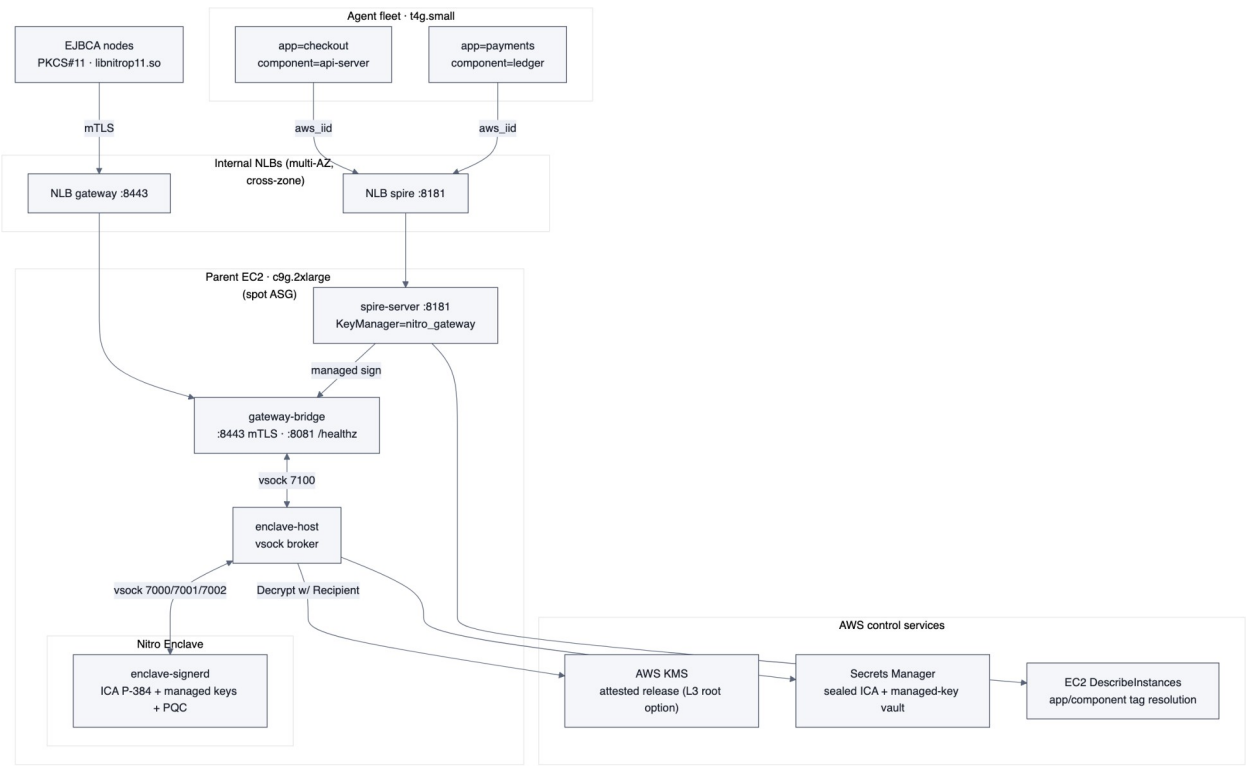
distinguishable, and no identity is inferred.

9 Deployment topology

Parent EC2 (c9g.2xlarge): gateway-bridge (:8443 mTLS, :8081 /healthz), enclave-host (vsock broker), spire-server (:8181). Enclave: enclave-signerd (N vCPU, ICA + managed keys). Agents (t4g.small): app=checkout/component=api-server, app=payments/component=ledger. AWS: KMS (attested release), Secrets Manager (sealed ICA + managed-key vault), EC2 DescribeInstances (tag resolution). Two internal NLBs planned (spire :8181, gateway :8443).

vsock ports

creds 7000 · KMS proxy 7001 · sealed-key vault 7002 · signer 7100 · ParentCID 3



Deployment — parent EC2 + enclave, agent fleet, AWS anchors, two internal NLBs.

10 Validation & load results (measured on AWS)

All numbers below were measured on a single c9g.2xlarge in us-east-1 with 4 of the 8 vCPU carved to the enclave, at concurrency 512 unless noted, with 0 errors.

Three-level throughput

Path	Peak	Bottleneck
------	------	------------

Raw enclave sign (HSM primitive)	~8,600/sec (ecdsa-p384)	enclave
SPIRE agent→server→gateway (SVID)	~236 SVIDs/sec	SPIRE server
EJBCA cert→gateway (issuance)	~895 certs/sec	EJBCA

Enclave per-cipher ceiling (conc 512, 0 errors)

Cipher	signs/sec
ECDSA P-256	4,374
ECDSA P-384	7,530
RSA-2048	2,888
RSA-4096	497
ML-DSA-65 (PQC)	6,261
ML-KEM-768	7,912
ML-KEM-1024	8,478
Hybrid P-384+ML-DSA-65	6,207

EJBCA issuance vs concurrency (ecdsa-p256, 0 errors)

Concurrency	certs/sec
16	767
32	895 (peak)
64	805

Takeaway — the signer is never the bottleneck

At ~8,600 ecdsa-p384 signs/sec on half a box, the enclave carries ~10–36× headroom over both issuance tiers. Scale product throughput by adding EJBCA nodes and SPIRE servers behind the NLB, not by scaling the signer.

- **The SPIRE `x509 mint` CLI figure (~78/sec)** is process-spawn-bound (each mint forks a CLI and reconnects), not the server ceiling — the real server-path number is ~236/sec.
- **The ~8,600/sec headline** is the raw sign primitive (random digest → signature) and deliberately bypasses SPIRE and EJBCA — it is the enclave HSM ceiling, not an issuance rate.

11 Security controls & custody statement

- **Key never leaves the enclave** — The signing key is generated, held, and used only inside the enclave; it never leaves that boundary.
- **Attested release bound to PCR0** — kms:Decrypt release is bound to the measured PCR0 via the KMS key policy, so only the exact enclave image can obtain the key.

- **Sealed blobs inert in Secrets Manager** — sealed managed keys persist as inert AES-256-GCM ciphertext in Secrets Manager, restorable only on an attested enclave with the same PCRO.
- **mTLS between EJBCA/SPIRE and the gateway** — EJBCA (via the .so) and SPIRE (via the KeyManager) speak mutual TLS to the gateway on :8443; the parent terminates mTLS but only sees digests and signatures.
- **Enclave-aware health** — gateway-bridge exposes an enclave-aware /healthz; a wedged enclave returns 503 so the NLB drains it and the ASG auto-replaces the box.
- **PQC-native** — ML-DSA-65, ML-KEM-768/1024 and hybrid (P-384+ML-DSA) signing are available in software today, without waiting on HSM firmware.

12 Next — horizontal scaling

Today's ceilings are single-box and service-bound, and both tiers are NLB-scalable. The plan puts each tier behind its own internal NLB with an autoscaling group: a shared RDS/Aurora datastore makes the SPIRE server tier stateless, and the Secrets Manager sealed-ICA and managed-key vault makes the gateway/enclave tier stateless, so each NLB can spray load across any box. The full two-NLB design, task list, and load-test matrix live in `spire/NLB-SCALING-PLAN.md` and are not duplicated here.